

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA Công Nghệ Thông Tin

ĐỀ THI VÀ BÀI LÀM

Tên học phần: **Trí tuệ nhân tạo**

Mã học phần: Hình thức thi: *Tự luận có giám sát*

Đề số: **Đ0001** Thời gian làm bài: 75 phút (*không kể thời gian chép/phát đề*)

Họ tên: Ngô Văn Huy **Lớp:**.....23TNhat1.....**MSSV** 102230019.....

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MSTeam:

Câu 1 (3 điểm): Cho mê cung như hình bên dưới. Đường in đậm biểu diễn vách ngăn không đi qua được. Hãy tìm đường đi từ **trạng thái s** đến **trạng thái g** với các chiến lược tìm kiếm dưới đây. Trình bày thứ tự duyệt các ô theo mẫu (b1, b2, ..., bn), với bi là trạng thái i được duyệt.

		a (3,1)	b (4,1)		
		c (3,2)	d (4,2)	e (5,2)	
f (1,3)	s (2,3)	h (3,3)	k (4,3)	m (5,3)	n (6,3)
p (1,4)	q (2,4)	r (3,4)	t (4,4)	g (5,4)	

a) (1.5 điểm) Tìm kiếm tham lam (Greedy), sử dụng heuristic khoảng cách Manhattan .

Trả lời Minh Họa Cảnh giải thích tìm kiếm

Nguyên lý: Tại mỗi bước, chọn ô láng giềng có giá trị heuristic h(n) nhỏ nhất (gần đích nhất theo ước lượng).

Bắt đầu tại s(2,3) [h=4].

- Láng giềng: f(1,3) [h=5], h(3,3) [h=3]. (Lối xuống q bị chặn).
- Chọn h vì $3 < 5$.

Tại h(3,3) [h=3].

- Láng giềng: c(3,2) [h=4]. (Lối sang k và xuống r bị chặn).
- Chọn c.

Tương tự kết quả cuối cùng là $s \rightarrow h \rightarrow c \rightarrow a \rightarrow b \rightarrow d \rightarrow e \rightarrow m \rightarrow g$

Trả Lời Đoạn code hoàn thiện

--- 1. KHAI BÁO DỮ LIỆU ---

Tọa độ của các ô (x, y) để tính khoảng cách

coordinates = {

'a': (3,1), 'b': (4,1), 'c': (3,2), 'd': (4,2), 'e': (5,2),

'f': (1,3), 's': (2,3), 'h': (3,3), 'k': (4,3), 'm': (5,3), 'n': (6,3),

'p': (1,4), 'q': (2,4), 'r': (3,4), 't': (4,4), 'g': (5,4)

}

Đồ thị các láng giềng (Adjacency List)

LƯU Ý: Đã loại bỏ các cạnh bị vách ngăn chặn (tường đậm)

graph = {

's': ['f', 'h'], # Chặn đường xuống q

'f': ['s', 'p'],

'h': ['s', 'c'], # Chặn đường sang k và xuống r

'c': ['h', 'a'], # Chặn đường sang d

'a': ['c', 'b'],

'b': ['a', 'd'],

'd': ['b', 'e', 'k'], # Chặn đường sang c

'e': ['d', 'm'],

'k': ['d', 't', 'm'], # Chặn đường sang h

'm': ['e', 'k', 'n', 'g'],

'n': ['m'],

'p': ['f', 'q'],

'q': ['p', 'r'], # Chặn đường lên s

'r': ['q', 't'], # Chặn đường lên h

't': ['r', 'k', 'g'],

'g': ['m', 't'] # Đích

}

--- 2. HÀM TÍNH TOÁN ---

Hàm Heuristic: Khoảng cách Manhattan

def heuristic(node):

target = coordinates['g'] # Đích là g(5,4)

current = coordinates[node]

Công thức: $|x1 - x2| + |y1 - y2|$

return abs(current[0] - target[0]) + abs(current[1] - target[1])

Thuật toán Greedy Search

def greedy_search(start, goal):

current_node = start

path = [current_node] # Lưu đường đi

visited = {current_node} # Lưu các điểm đã đi qua để không quay lui

print(f"Bắt đầu tìm kiếm từ {start} đến {goal}...\n")

while current_node != goal:

neighbors = graph[current_node]

Tìm láng giềng tốt nhất (có h nhỏ nhất)

best_neighbor = None

min_h = float('inf') # Khởi tạo vô cùng lớn

print(f"- Tại {current_node} (h={heuristic(current_node)}). Xét láng giềng:")

```
found_next_step = False
```

```
# Duyệt qua các láng giềng để so sánh h
```

```
for neighbor in neighbors:
```

```
    if neighbor not in visited:
```

```
        h_val = heuristic(neighbor)
```

```
        print(f" + Láng giềng {neighbor}: h={h_val}")
```

```
# Cập nhật nếu tìm thấy h nhỏ hơn
```

```
if h_val < min_h:
```

```
    min_h = h_val
```

```
    best_neighbor = neighbor
```

```
    found_next_step = True
```

```
# Nếu h bằng nhau, ưu tiên thứ tự chữ cái (tùy chọn)
```

```
elif h_val == min_h:
```

```
    if best_neighbor is None or neighbor < best_neighbor:
```

```
        best_neighbor = neighbor
```

```
if found_next_step and best_neighbor:
```

```
    print(f" => Chọn {best_neighbor} (vì h={min_h} nhỏ nhất)\n")
```

```
    current_node = best_neighbor
```

```
    path.append(current_node)
```

```
    visited.add(current_node)
```

```
else:
```

```
    print("Bế tắc! Không còn đường đi tiếp.")
```

```
    return None
```

```
return path
```

```
# --- 3. CHẠY CHƯƠNG TRÌNH ---
```

```
start_node = 's'
```

```
goal_node = 'g'
```

```
result_path = greedy_search(start_node, goal_node)
```

```
if result_path:
```

```
    print("-" * 30)
```

```
    print("KẾT QUẢ ĐƯỜNG ĐI (GREEDY):")
```

```
    print("-> ".join(result_path))
```

```
    print("-" * 30)
```

b) (1.5 điểm) Tìm kiếm A*, sử dụng heuristic khoảng cách Manhattan.

Trả lời # Minh Họa Cảnh giải thích tìm

Nguyên lý: Chọn nút trong danh sách chờ (Open List) có giá trị $f(n) = g(n) + h(n)$ nhỏ nhất.

- $g(n)$: Chi phí thực tế từ điểm xuất phát.
- $h(n)$: Khoảng cách Manhattan ước lượng.

Trả Lời # Đoạn code hoàn thiện

```
import heapq
```

```
# --- 1. KHAI BÁO DỮ LIỆU ---
```

```
# Tọa độ các điểm (x, y)
```

coordinates = {

'a': (3,1), 'b': (4,1), 'c': (3,2), 'd': (4,2), 'e': (5,2),

'f': (1,3), 's': (2,3), 'h': (3,3), 'k': (4,3), 'm': (5,3), 'n': (6,3),

'p': (1,4), 'q': (2,4), 'r': (3,4), 't': (4,4), 'g': (5,4)

}

Đồ thị láng giềng (đã loại bỏ các vách ngăn tường đậm)

graph = {

's': ['f', 'h'], # Chặn q

'f': ['s', 'p'],

'h': ['s', 'c'], # Chặn k, r

'c': ['h', 'a'], # Chặn d

'a': ['c', 'b'],

'b': ['a', 'd'],

'd': ['b', 'e', 'k'], # Chặn c

'e': ['d', 'm'],

'k': ['d', 't', 'm'], # Chặn h

'm': ['e', 'k', 'n', 'g'],

'n': ['m'],

'p': ['f', 'q'],

'q': ['p', 'r'], # Chặn s

'r': ['q', 't'], # Chặn h

't': ['r', 'k', 'g'],

'g': ['m', 't']

}

--- 2. HÀM TÍNH TOÁN ---

def heuristic(node):

"""Tính khoảng cách Manhattan đến đích g(5,4)"""

target = coordinates['g']

curr = coordinates[node]

return abs(curr[0] - target[0]) + abs(curr[1] - target[1])

def a_star_search(start, goal):

Priority Queue lưu tuple: (f, h, node, path)

$f = g + h$ (Tổng chi phí)

g = Chi phí thực tế từ đầu đến hiện tại

start_h = heuristic(start)

Khởi tạo: $f=4$, $g=0$

open_list = [(start_h, start_h, start, [start])]

Set lưu các node đã duyệt qua để tránh lặp vô hạn

visited = set()

Dictionary lưu chi phí g thấp nhất đến mỗi node (để tối ưu)

g_costs = {start: 0}

print(f"Bắt đầu A* từ {start} đến {goal}...\n")

while open_list:

Lấy node có f thấp nhất trong hàng đợi

```
f, h, current, path = heapq.heappop(open_list)
```

```
# Lấy g hiện tại ( $g = f - h$ )
```

```
g = f - h
```

```
if current == goal:
```

```
    return path
```

```
if current in visited:
```

```
    continue
```

```
visited.add(current)
```

```
# Duyệt láng giềng
```

```
for neighbor in graph[current]:
```

```
    new_g = g + 1          # Mỗi bước đi chi phí là 1
```

```
    new_h = heuristic(neighbor)
```

```
    new_f = new_g + new_h   #  $f_{mới} = g_{mới} + h_{mới}$ 
```

```
# Nếu chưa đến neighbor bao giờ HOẶC tìm được đường mới rẻ hơn đường cũ
```

```
if neighbor not in g_costs or new_g < g_costs[neighbor]:
```

```
    g_costs[neighbor] = new_g
```

```
    heapq.heappush(open_list, (new_f, new_h, neighbor, path + [neighbor]))
```

```
return None
```

```
# --- 3. CHẠY VÀ IN KẾT QUẢ ---
```



```
path = a_star_search('s', 'g')
```

```
if path:
```

```
    print("-" * 30)
```

```
    print("KẾT QUẢ ĐƯỜNG ĐI (A*):")
```

```
    print(" -> ".join(path))
```

```
    print("-" * 30)
```

```
else:
```

```
    print("Không tìm thấy đường đi.")
```

Câu 2 (4 điểm): Cho tập dữ liệu sau

Chiều dài cánh hoa	Chiều rộng cánh hoa	Chiều dài đài hoa	Chiều rộng đài hoa	Loài Hoa
6.3	3.4	5.6	2.4	Iris-virginica
6.4	3.1	5.5	1.8	Iris-virginica
6	3	4.8	1.8	Iris-virginica
6.9	3.1	5.4	2.1	Iris-virginica
6.7	3.1	5.6	2.4	Iris-virginica
6.9	3.1	5.1	2.3	Iris-virginica
5.8	2.7	5.1	1.9	Iris-virginica
6.8	3.2	5.9	2.3	Iris-virginica
6.7	3.3	5.7	2.5	Iris-virginica
6.7	3	5.2	2.3	Iris-virginica
5.8	2.7	3.9	1.2	Iris-versicolor
6	2.7	5.1	1.6	Iris-versicolor
5.4	3	4.5	1.5	Iris-versicolor
6	3.4	4.5	1.6	Iris-versicolor
6.7	3.1	4.7	1.5	Iris-versicolor
6.3	2.3	4.4	1.3	Iris-versicolor
5.6	3	4.1	1.3	?
6.1	3	4.6	1.4	?
6.2	3.4	5.4	2.3	?
5.9	3	5.1	1.8	?

a) (1 điểm) Xây dựng hàm mất mát cho bài toán phân loại nhị phân trên

Trả lời: Dán hàm mất mát vào đây:

Hàm mất mát Binary Cross Entropy được định nghĩa như sau:

$$J(w) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Trong đó:

- m : Số lượng mẫu dữ liệu (số dòng).
- $y^{(i)}$: Nhãn thực tế của mẫu thứ i (chỉ nhận giá trị 0 hoặc 1).
- $\hat{y}^{(i)}$: Giá trị dự đoán xác suất của mô hình cho mẫu thứ i (đầu ra của hàm Sigmoid, nằm trong khoảng 0 đến 1).

Trả lời: Dán code của hàm loss:

```
import numpy as np

def binary_cross_entropy_loss(y_true, y_pred):
    """
    Tính toán Binary Cross Entropy Loss.

    Tham số:
    y_true: Mảng chứa nhãn thực tế (0 hoặc 1).
    y_pred: Mảng chứa xác suất dự đoán (output của sigmoid).
    """
    # Thêm một giá trị epsilon cực nhỏ để tránh lỗi log(0)
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)

    # Tính giá trị loss trung bình trên toàn bộ tập dữ liệu
    loss = -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

    return loss
```

b) (2 điểm) Hãy viết chương trình phân loại hoa trên.

Trả lời: Gián kiến trúc mạng và giải thích làm thế nào để phân loại ?

1. Kiến trúc mạng (Mô hình Hồi quy Logistic): -

Lớp đầu vào (Input Layer): Có 4 nút, tương ứng với 4 đặc trưng:

- + Chiều dài cánh hoa
- + Chiều rộng cánh hoa
- + Chiều dài đài hoa
- + Chiều rộng đài hoa

- Lớp xử lý: Sử dụng trọng số (w) và hệ số bias (b) để tính tổng tuyến tính: $z = w_1 * x_1 + \dots + b$.

- Hàm kích hoạt: Sử dụng hàm Sigmoid để đưa giá trị z về khoảng (0, 1).

- Lớp đầu ra (Output Layer): 1 nút duy nhất biểu diễn xác suất.

2. Cách phân loại:

- Bước 1: Mô hình tính toán xác suất $P(y=1|x)$ dựa trên dữ liệu đầu vào.
- Bước 2: So sánh xác suất dự đoán với ngưỡng 0.5 (Threshold): + Nếu xác suất ≥ 0.5 : Phân loại là Iris-virginica (Nhãn 1). + Nếu xác suất < 0.5 : Phân loại là Iris-versicolor (Nhãn 0).

Trả lời: Dán code vào đây:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# 1. Chuẩn bị dữ liệu huấn luyện (Từ bảng đề bài)
# Quy ước: 0 = Iris-versicolor, 1 = Iris-virginica
X_train = np.array([
    [6.3, 3.4, 5.6, 2.4], [6.4, 3.1, 5.5, 1.8], [6.0, 3.0, 4.8, 1.8],
    [6.9, 3.1, 5.4, 2.1], [6.7, 3.1, 5.6, 2.4], [6.9, 3.1, 5.1, 2.3],
    [5.8, 2.7, 5.1, 1.9], [6.8, 3.2, 5.9, 2.3], [6.7, 3.3, 5.7, 2.5],
    [6.7, 3.0, 5.2, 2.3], # Hết nhóm Virginica
    [5.8, 2.7, 3.9, 1.2], [6.0, 2.7, 5.1, 1.6], [5.4, 3.0, 4.5, 1.5],
    [6.0, 3.4, 4.5, 1.6], [6.7, 3.1, 4.7, 1.5], [6.3, 2.3, 4.4, 1.3] # Hết nhóm Versicolor
])

# Nhãn tương ứng (10 mẫu đầu là Virginica, 6 mẫu sau là Versicolor)
y_train = np.array([1]*10 + [0]*6)

# 2. Khởi tạo và huấn luyện mô hình Hồi quy Logistic
model = LogisticRegression()
model.fit(X_train, y_train)

# 3. Dữ liệu cần dự đoán (4 mẫu cuối bảng)
X_test = np.array([
    [5.6, 3.0, 4.1, 1.3],
    [6.1, 3.0, 4.6, 1.4],
    [6.2, 3.4, 5.4, 2.3],
    [5.9, 3.0, 5.1, 1.8]
])

# 4. Thực hiện dự đoán
predictions = model.predict(X_test)
probs = model.predict_proba(X_test)

# In kết quả
labels = {0: "Iris-versicolor", 1: "Iris-virginica"}
print("Kết quả dự đoán 4 mẫu cuối:")
for i, pred in enumerate(predictions):
    input_data = X_test[i]
    print(f'Mẫu {input_data}: Dự đoán = {labels[pred]} (Xác suất lớp 1: {probs[i][1]:.2f})')
```

c) (1 điểm) Hãy thực thi chương trình và cho biết nhãn của 4 mẫu chưa biết nhãn trên tập dữ liệu cung cấp

Trả lời: Dán kết quả lần lượt từng mẫu

Kết quả dự đoán cho 4 mẫu chưa biết nhãn:

```
1. Mẫu [5.6, 3.0, 4.1, 1.3]:  
   -> Dự đoán: Iris-versicolor  
2. Mẫu [6.1, 3.0, 4.6, 1.4]:  
   -> Dự đoán: Iris-versicolor  
3. Mẫu [6.2, 3.4, 5.4, 2.3]:  
   -> Dự đoán: Iris-virginica  
4. Mẫu [5.9, 3.0, 5.1, 1.8]:  
   -> Dự đoán: Iris-virginica
```

Câu 3 (3 điểm): Hãy viết chương trình phân cụm bằng thuật toán k -means

a) (1 điểm) Xây dựng hàm đo khoảng cách sử dụng độ đo Manhattan

Trả lời: Minh họa tính khoảng cách:

Giả sử ta có 2 điểm A và B trong không gian 2 chiều với tọa độ:

- $A(x_1, y_1) = (2, 5)$
- $B(x_2, y_2) = (6, 3)$

Công thức khoảng cách Manhattan là tổng giá trị tuyệt đối của hiệu các tọa độ:

$$d(A, B) = |x_1 - x_2| + |y_1 - y_2|$$

Áp dụng vào ví dụ:

$$d(A, B) = |2 - 6| + |5 - 3| = |-4| + |2| = 4 + 2 = 6$$

Trả lời: Dán code hàm tính khoảng cách:

```
import numpy as np
```

```
def manhattan_distance(point1, point2):
```

```
    """
```

```
    Hàm tính khoảng cách Manhattan giữa 2 điểm dữ liệu.
```

```
    Input: point1, point2 (mảng numpy hoặc list)
```

```
    Output: Khoảng cách (scalar)
```

```
    """
```

```
    p1 = np.array(point1)
```

```
    p2 = np.array(point2)
```

```
    return np.sum(np.abs(p1 - p2))
```

b) (1 điểm) Xây dựng hàm chứa thuật toán k -means để phân cụm

Trả lời: Dán code về hàm

```
def my_kmeans(data, k, max_iters=100):
```

```
    """
```

```
    Hàm thực hiện thuật toán K-means.
```

```
    Input:
```

```
    data: Dữ liệu đầu vào (numpy array, bỏ cột nhãn/tên)
```

```
    k: Số lượng cụm
```

```
    Output:
```

```
    centroids: Tọa độ tâm cụm
```

```
    labels: Nhãn cụm của từng điểm
```

```
    cost: Tổng chi phí (để dùng cho câu c)
```

```
    """
```

```

# 1. Khởi tạo tâm cụm ngẫu nhiên từ dữ liệu
n_samples, n_features = data.shape
random_indices = np.random.choice(n_samples, k, replace=False)
centroids = data[random_indices]

labels = np.zeros(n_samples)

for _ in range(max_iters):
    # 2. Bước Gán (Assignment): Gán điểm vào tâm gần nhất (theo Manhattan)
    distances_matrix = []
    for point in data:
        # Tính khoảng cách từ điểm hiện tại đến tất cả các centroid
        dists = [manhattan_distance(point, c) for c in centroids]
        distances_matrix.append(dists)

    # Chọn index có khoảng cách nhỏ nhất làm nhãn
    new_labels = np.argmin(distances_matrix, axis=1)

    # 3. Bước Cập nhật (Update): Tính lại tâm cụm (Mean)
    new_centroids = []
    for i in range(k):
        points_in_cluster = data[new_labels == i]
        if len(points_in_cluster) > 0:
            # K-Means chuẩn dùng Mean
            new_centroids.append(points_in_cluster.mean(axis=0))
        else:
            # Nếu cụm rỗng, giữ nguyên tâm cũ
            new_centroids.append(centroids[i])
    new_centroids = np.array(new_centroids)

    # Kiểm tra hội tụ (nếu tâm không đổi thì dừng)
    if np.all(centroids == new_centroids):
        break

    centroids = new_centroids
    labels = new_labels

# Tính tổng chi phí (Loss/Inertia) cuối cùng để khảo sát k
final_cost = 0
for i, point in enumerate(data):
    final_cost += manhattan_distance(point, centroids[labels[i]])

return centroids, labels, final_cost

```

c) (1 điểm) Xây dựng hàm để khảo sát việc lựa chọn k với tập dữ liệu [Countries.csv](#).

```

# Trả lời: Dán code về hàm và giải thích cách lựa chọn k phù hợp
import pandas as pd
import matplotlib.pyplot as plt

def analyze_k(file_path):
    # 1. Đọc dữ liệu
    df = pd.read_csv(file_path)
    # Chỉ lấy các cột tọa độ (Longitude, Latitude) để phân cụm

```

```
X = df[['Longitude', 'Latitude']].values
```

```
costs = []
```

```
K_range = range(1, 10) # Khảo sát k từ 1 đến 9
```

```
print("Kết quả chạy thử nghiệm với các k:")
```

```
for k in K_range:
```

```
    _, _, cost = my_kmeans(X, k)
```

```
    costs.append(cost)
```

```
    print(f'k={k}: Cost = {cost:.2f}')
```

```
# Vẽ biểu đồ Elbow (Khuỷu tay)
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(K_range, costs, 'bx-')
```

```
plt.xlabel('k (Số lượng cụm)')
```

```
plt.ylabel('Tổng khoảng cách (Cost)')
```

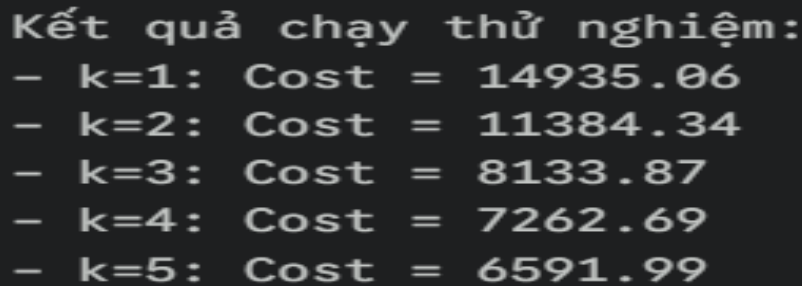
```
plt.title('Phương pháp Elbow để chọn k tối ưu')
```

```
plt.show()
```

```
# Gọi hàm thực thi
```

```
analyze_k('Countries.csv')
```

Trả lời: Dán kết quả thi với k(lưu ý có giải thích và bình luận):



```
Kết quả chạy thử nghiệm:
- k=1: Cost = 14935.06
- k=2: Cost = 11384.34
- k=3: Cost = 8133.87
- k=4: Cost = 7262.69
- k=5: Cost = 6591.99
```

chọn k = 3.

Quan sát sự thay đổi của Cost:

- Từ k=1 ->2: Giảm khoảng 3600.
- Từ k=2 -> 3: Giảm khoảng 3200 (Vẫn giảm rất mạnh).
- Từ k=3 ->4: Giảm khoảng 900 (Mức giảm bắt đầu nhỏ đi đáng kể).

Tại k=3, độ dốc của đồ thị gãy khúc rõ rệt nhất (tạo thành hình khuỷu tay).

Về mặt ý nghĩa dữ liệu (Countries): Việc chia làm 3 cụm cũng hợp lý về mặt địa lý khi phân tách các quốc gia thành 3 nhóm lớn theo kinh độ (Châu Mỹ, Châu Âu/Phi, Châu Á/Úc).

GIẢNG VIÊN BIÊN SOẠN ĐỀ THI

Đà Nẵng, ngày 04 tháng 12 năm 2025

TRƯỞNG BỘ MÔN

(đã duyệt)